Autor: Iván García Santillán

Algoritmos predictivos de clasificación: Regresión logística

Introducción

Es un método de aprendizaje automático supervisado utilizado para resolver problemas de clasificación binaria o multinomial (más de dos clases). Aunque su nombre incluye la palabra "regresión", en realidad se trata de un algoritmo de clasificación.

Se utiliza comúnmente en una variedad de aplicaciones, como la detección de *spam* de correo electrónico, diagnósticos médicos, clasificación de documentos, entre otros. Por ejemplo, en medicina, es fundamental para predecir la probabilidad de ocurrencia de ciertas enfermedades, como la diabetes o el cáncer, basándose en factores de riesgo.

En el sector financiero, se aplica en la evaluación del riesgo crediticio, determinando la probabilidad de que un cliente incumpla con sus pagos.

En *marketing*, se utiliza para predecir la probabilidad de que un cliente compre un producto o responda a una campaña específica. Además, la regresión logística es esencial en el campo de la investigación social y política para analizar la influencia de diversas variables sobre resultados binarios, como la decisión de votar por un candidato.

Su capacidad para proporcionar probabilidades y su facilidad de interpretación la hacen una herramienta valiosa para la toma de decisiones basada en datos. Aunque también se deben considerar algunas desventajas como la sensibilidad a datos atípicos (*outliers*), afectando significativamente la estimación de los coeficientes y distorsionando el modelo. Además, el problema de la multicolinealidad donde hay alta correlación entre las variables independientes (predictoras), así como el problema con datos desbalanceados donde puede desarrollar un sesgo hacia la clase más frecuente.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

A continuación, se detallan el funcionamiento del algoritmo y un ejemplo en Python, Keras y TensorFlow para la predicción del cáncer de mama utilizando un data set público.

Contenido

El nombre de regresión logística proviene de la función **logística** (también conocida como función **sigmoide**) que utiliza para modelar la relación entre las variables de entrada (características) y la probabilidad de que una instancia pertenezca a una clase específica, como se indica en la Figura 16.

Figura 16. Función logística (sigmoide)

$$f(x) = \frac{1}{1+e^{-x}}$$

La función logística tiene la siguiente forma:

$$P(Y=1) = \frac{1}{1+e^{-(a_0+a_1X_1+a_2X_2+\ldots+a_nX_n)}}$$

Donde:

- *P(Y=1)* es la probabilidad de pertenecer a la clase 1.
- *e* es la base del logaritmo natural.
- a_0, a_1, a_2, …, a_n son coeficientes que se aprenden (ajustan) durante el entrenamiento.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

- $X_1, X_2, \ldots, X_n$ son las variables de entrada.

Durante la fase de entrenamiento, el algoritmo de regresión logística ajusta los coeficientes $a_0, a_1, a_2, \ldots, a_n$ de manera que el modelo se ajuste mejor a los datos de entrenamiento.

Una vez que el modelo está entrenado, se puede utilizar para hacer predicciones sobre nuevos datos, es decir, el modelo calcula la probabilidad de que esa instancia pertenezca a una de las clases. Si la probabilidad calculada es mayor que el umbral (generalmente 0.5), se clasifica como la clase positiva (1); de lo contrario, se clasifica como la clase negativa (0) en un problema de clasificación binaria.

A continuación, se presenta un ejemplo del algoritmo de regresión logística en Python, Keras y TensorFlow para la predicción del cáncer de mama (benigno=0, maligno=1) utilizando el conjunto de datos Breast Cancer Wisconsin (Diagnostic) disponible en la librería scikit-learn.

```
# Logistic regression for breast cancer
# importa librerías necesarias
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from sklearn.linear_model import LogisticRegression

# Carga el conjunto de datos Breast Cancer
dataset = load_breast_cancer()
X = dataset.data # 569x30
y = dataset.target # 569x1

# Divide el conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Normaliza los datos para que tengan una escala similar
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
X_test = scaler.transform(X_test)

# Crea y entrena el modelo de regresión logistica
model = LogisticRegression(multi_class='auto', solver='lbfgs', max_iter=100)
model.fit(X_train, y_train)

# Realiza predicciones usando el conjunto de prueba
y_pred = model.predict(X_test)

# Convierte las probabilidades en etiquetas binarias (0 o 1)
y_pred = (y_pred > 0.5)

# Muestra el informe de evaluación del modelo entrenado
print(classification_report(y_test, y_pred))

# Matriz de confusión:
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
print("confusion matrix: \n", cm)
# gráfica de la CM
plt.figure(figsize = (8,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Prediction', fontsize = 12)
plt.ylabel('Real', fontsize = 12)
plt.show()

# Exactitud:
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_test, y_pred)
print("accuracy: ", acc)

# Sensibilidad:
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_pred)
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

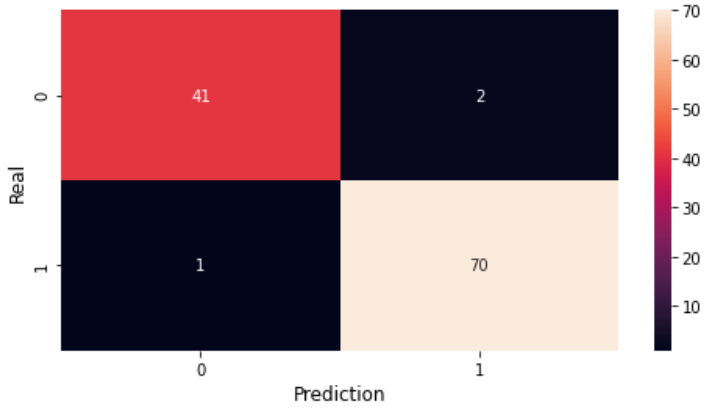
```
print("recall: ", recall)

# Precisión:
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_pred)
print("precision: ", precision)

# Especificidad
# 'specificity' is just a special case of 'recall'.
# specificity is the recall of the negative class
specificity = recall_score(y_test, y_pred, pos_label=0)
print("specificity: ", specificity)

# Puntuación F1:
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
print("f1 score: ", f1)

# Área bajo la curva:
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y_test, y_pred)
print("auc: ", auc)

# Curva ROC
from sklearn.metrics import roc_curve
plt.figure()
lw = 2
plt.plot(roc_curve(y_test, y_pred)[0], roc_curve(y_test, y_pred)[1],
color='darkorange',lw=lw, label='ROC curve (area = %0.2f)' %auc)
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```

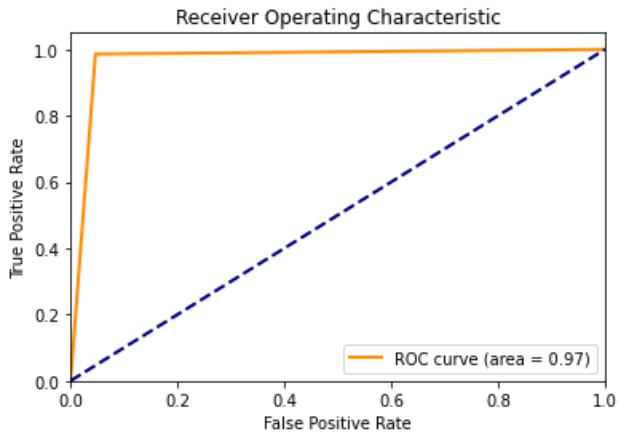
Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

```
# R Score (R^2 coefficient of determination)
from sklearn.metrics import r2_score
R = r2_score(y_test, y_pred)
print("R2: ", R)
```

Los resultados del modelo de regresión logística se presentan a continuación:

accuracy: 0.97
recall: 0.98
precision: 0.97
specificity: 0.95
f1 score: 0.97
auc: 0.97
R2: 0.88





Un clasificador adecuado es aquel que alcanza una exactitud (*accuracy*) superior al 85 %. Por lo que, este modelo de regresión logística entrenado cumple con este criterio de calidad.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Adicionalmente, tenga en cuenta que se pueden ajustar varios parámetros del algoritmo (como multi_class, solver, max_iter, etc.) para obtener un modelo óptimo. Revise la documentación (ayuda en línea https://scikit-learn.org/stable/supervised_learning.html) del algoritmo para más detalles y experimente con ellos.

Finalmente, aunque la regresión logística es un modelo fundamental en la caja de herramientas de análisis predictivo, como cualquier otra técnica, presenta tanto ventajas como desventajas que se resumen a continuación:

Ventajas de la Regresión Logística:

Interpretabilidad: una de las principales fortalezas de la regresión logística es su interpretabilidad fácil y directa. Los coeficientes del modelo pueden ser interpretados para cada uno de los predictores independientes, proporcionando una clara comprensión del impacto de cada variable en la probabilidad del resultado.

Eficiencia computacional: la regresión logística es menos intensiva en términos computacionales en comparación con modelos más complejos como las redes neuronales, lo que la hace relativamente rápida de entrenar.

Buen rendimiento con variables categóricas y numéricas: el algoritmo funciona bien con una combinación de características numéricas y categóricas.

Probabilidades de salida: a diferencia de muchos otros clasificadores, la regresión logística proporciona una probabilidad cuantitativa para las predicciones, lo que puede ser crucial para ciertas decisiones donde el umbral de decisión necesita ser ajustado.

Bueno para modelos lineales: funciona muy bien si la relación entre la variable dependiente y las independientes es aproximadamente lineal, común en problemas de complejidad baja y media.

Desventajas de la Regresión Logística:

No es adecuada para relaciones no lineales: el algoritmo no puede manejar relaciones no lineales sin transformación de las variables independientes,

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

común en problemas complejos. El término modelar “relaciones no lineales" se refiere a la capacidad de un modelo estadístico o de aprendizaje automático para capturar y representar relaciones entre variables que no son simplemente proporcionales o directas. En una relación no lineal, los cambios en una variable no producen cambios proporcionales y predecibles en otra variable, lo que significa que la relación entre variables no puede ser descrita adecuadamente por una línea recta, sino por otro tipo de relación como cuadrática, exponencial, logarítmica, sinusoidal, etc.

Sensibilidad a datos no balanceados: si las clases no están aproximadamente igualmente representadas, la regresión logística puede desarrollar un sesgo hacia la clase dominante. Esto puede requerir técnicas de muestreo o ponderación para corregir.

Influencia de valores atípicos: como en muchos modelos lineales, los valores atípicos pueden tener un efecto desproporcionadamente alto en la estimación del modelo, lo que puede llevar a estimaciones y predicciones pobres.

Multicolinealidad: problemas de multicolinealidad pueden surgir cuando dos o más variables predictoras son altamente correlacionadas. Ésto no sólo influye en la precisión de la estimación de los coeficientes, sino también en la interpretación de estos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.

Conclusiones

En esta lectura se ha revisado los fundamentos del algoritmo de regresión logística para abordar problemas de clasificación binaria utilizando Python y TensorFlow-Keras.

La regresión logística es una herramienta poderosa y flexible para problemas de clasificación, especialmente cuando la claridad de la interpretación y la velocidad de cálculo son preocupaciones primordiales. Sin embargo, su aplicación debe ser considerada viendo que las limitaciones del modelo no afecten negativamente los resultados del análisis. Es importante experimentar y ajustar varios parámetros del algoritmo para obtener un modelo óptimo.

Por lo que, se motiva a los estudiantes a revisar material y ejemplos adicionales disponibles libremente en Internet para profundizar en la comprensión del algoritmo y su aplicación eficaz en diferentes problemas de variada complejidad.

Bibliografía

Berzal, Fernando (2019). *Redes neuronales & Deep Learning*. USA: Independently published.

Gerón, A. (2023). *Aprende Machine Learning con Scikit-Learn, Keras y TensorFlow: Conceptos, herramientas y técnicas para conseguir sistemas inteligentes.* Tercera Edición. Anaya Multimedia.

Torres, J. (2020). *Python Deep Learning: Introducción práctica con Keras y TensorFlow 2*. Marcombo.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG Campus.